"IMAGE CLASSIFICATION USING VISION TRANSFORMER (ViT)

FOR PLANT DISEASE DETECTION"

Problem Statement : Current plant disease detection methods lack scalability, global context understanding, and interpretability, especially on small datasets.

In [13]:
```
import os
import torch
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image

os.environ["TOKENIZERS_PARALLELISM"] = "false"

# Switch to CPU — avoids MPS .view() bug
device = torch.device("cpu")
print(f"Using device: {device}")
print(f"PyTorch version: {torch.__version__}")
```

```
Using device: cpu
PyTorch version: 2.5.1
```

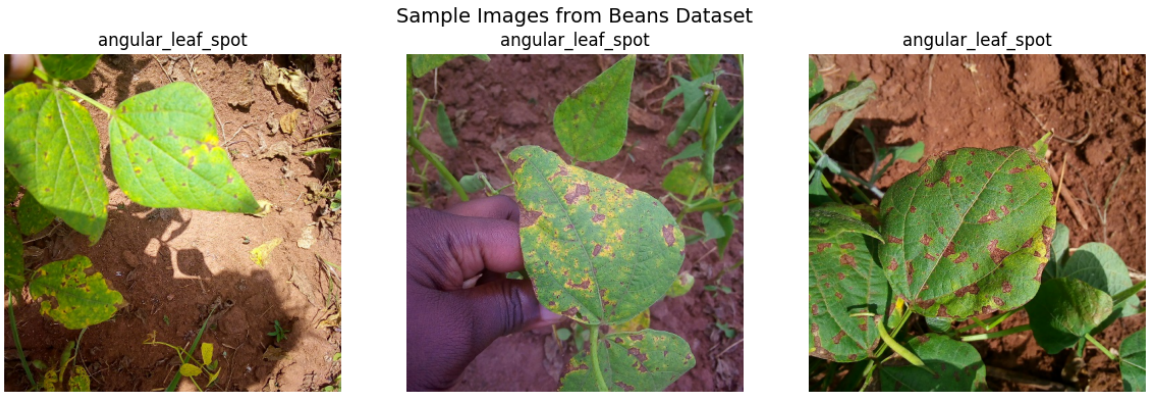
In [15]:
```
from datasets import load_dataset

dataset = load_dataset("beans")
print(dataset)

# Preview class names
labels = dataset["train"].features["labels"].names
print(f"\nClasses: {labels}")
print(f"Train size: {len(dataset['train'])}")
print(f"Validation size: {len(dataset['validation'])}")
print(f"Test size: {len(dataset['test'])}")
```

```
/opt/anaconda3/envs/vit-project/lib/python3.10/site-packages/tqdm/au
to.py:21: TqdmWarning: IProgress not found. Please update jupyter an
d ipywidgets. See https://ipywidgets.readthedocs.io/en/stable/user_i
nstall.html
  from .autonotebook import tqdm as notebook_tqdm
```

```
DatasetDict({
    train: Dataset({
        features: ['image_file_path', 'image', 'labels'],
        num_rows: 1034
    })
    validation: Dataset({
        features: ['image_file_path', 'image', 'labels'],
        num_rows: 133
    })
    test: Dataset({
        features: ['image_file_path', 'image', 'labels'],
        num_rows: 128
    })
})

Classes: ['angular_leaf_spot', 'bean_rust', 'healthy']
Train size: 1034
Validation size: 133
Test size: 128
```

In [16]:

```
fig, axes = plt.subplots(1, 3, figsize=(12, 4))

for i, ax in enumerate(axes):
    sample = dataset["train"][i]
    ax.imshow(sample["image"])
    ax.set_title(labels[sample["labels"]], fontsize=12)
    ax.axis("off")

plt.suptitle("Sample Images from Beans Dataset", fontsize=14)
plt.tight_layout()
plt.show()
```



In [17]:

```
from transformers import ViTImageProcessor

processor = ViTImageProcessor.from_pretrained("google/vit-base-patc

def preprocess(batch):
    inputs = processor(
        images=[img.convert("RGB") for img in batch["image"]],
        return_tensors="pt"
    )
    inputs["labels"] = batch["labels"]
    return inputs
```

```
encoded_dataset = dataset.map(preprocess, batched=True, remove_colu
encoded_dataset.set_format("torch")

print("Preprocessing done!")
print(f"Sample keys: {list(encoded_dataset['train'][0].keys())}")
print(f"Pixel values shape: {encoded_dataset['train'][0]['pixel_val
```

```
Preprocessing done!
Sample keys: ['image_file_path', 'labels', 'pixel_values']
Pixel values shape: torch.Size([3, 224, 224])
```

In [18]:

```
from transformers import ViTForImageClassification

id2label = {i: label for i, label in enumerate(labels)}
label2id = {label: i for i, label in enumerate(labels)}

model = ViTForImageClassification.from_pretrained(
    "google/vit-base-patch16-224",
    num_labels=len(labels),
    id2label=id2label,
    label2id=label2id,
    ignore_mismatched_sizes=True
)

model = model.to(device)
print(f"Model loaded on: {next(model.parameters()).device}")
print(f"Total parameters: {sum(p.numel() for p in model.parameters(
```

```
You passed `num_labels=3` which is incompatible to the `id2label` ma
p of length `1000`.
Loading weights: 100%|▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰| 200/200 [00:00<00:00,
7299.77it/s]
ViTForImageClassification LOAD REPORT from: google/vit-base-patch16-
224
Key                 | Status    |
-------------------+----------+-------------------------------------
-----------------------------------------------------
classifier.weight | MISMATCH | Reinit due to size mismatch - ckpt: t
orch.Size([1000, 768]) vs model:torch.Size([3, 768])
classifier.bias   | MISMATCH | Reinit due to size mismatch - ckpt: t
orch.Size([1000]) vs model:torch.Size([3])

Notes:
- MISMATCH:     ckpt weights were loaded, but they did not match the
original empty weight shapes.
Model loaded on: cpu
Total parameters: 85,800,963
```

In [19]:

```
pip install accelerate
```

```
Requirement already satisfied: accelerate in /opt/anaconda3/envs/vit
-project/lib/python3.10/site-packages (1.13.0)
Requirement already satisfied: numpy>=1.17 in /opt/anaconda3/envs/vi
t-project/lib/python3.10/site-packages (from accelerate) (2.2.5)
Requirement already satisfied: packaging>=20.0 in /opt/anaconda3/env
s/vit-project/lib/python3.10/site-packages (from accelerate) (26.0)
Requirement already satisfied: psutil in /opt/anaconda3/envs/vit-pro
```

```
ject/lib/python3.10/site-packages (from accelerate) (7.2.2)
Requirement already satisfied: pyyaml in /opt/anaconda3/envs/vit-pro
ject/lib/python3.10/site-packages (from accelerate) (6.0.3)
Requirement already satisfied: torch>=2.0.0 in /opt/anaconda3/envs/v
it-project/lib/python3.10/site-packages (from accelerate) (2.5.1)
Requirement already satisfied: huggingface_hub>=0.21.0 in /opt/anaco
nda3/envs/vit-project/lib/python3.10/site-packages (from accelerate)
(1.10.1)
Requirement already satisfied: safetensors>=0.4.3 in /opt/anaconda3/
envs/vit-project/lib/python3.10/site-packages (from accelerate) (0.
7.0)
Requirement already satisfied: filelock>=3.10.0 in /opt/anaconda3/en
vs/vit-project/lib/python3.10/site-packages (from huggingface_hub>=
0.21.0->accelerate) (3.20.3)
Requirement already satisfied: fsspec>=2023.5.0 in /opt/anaconda3/en
vs/vit-project/lib/python3.10/site-packages (from huggingface_hub>=
0.21.0->accelerate) (2026.2.0)
Requirement already satisfied: hf-xet<2.0.0,>=1.4.3 in /opt/anaconda
3/envs/vit-project/lib/python3.10/site-packages (from huggingface_hu
b>=0.21.0->accelerate) (1.4.3)
Requirement already satisfied: httpx<1,>=0.23.0 in /opt/anaconda3/en
vs/vit-project/lib/python3.10/site-packages (from huggingface_hub>=
0.21.0->accelerate) (0.28.1)
Requirement already satisfied: tqdm>=4.42.1 in /opt/anaconda3/envs/v
it-project/lib/python3.10/site-packages (from huggingface_hub>=0.21.
0->accelerate) (4.67.3)
Requirement already satisfied: typer in /opt/anaconda3/envs/vit-proj
ect/lib/python3.10/site-packages (from huggingface_hub>=0.21.0->acce
lerate) (0.24.1)
Requirement already satisfied: typing-extensions>=4.1.0 in /opt/anac
onda3/envs/vit-project/lib/python3.10/site-packages (from huggingfac
e_hub>=0.21.0->accelerate) (4.15.0)
Requirement already satisfied: anyio in /opt/anaconda3/envs/vit-proj
ect/lib/python3.10/site-packages (from httpx<1,>=0.23.0->huggingface
_hub>=0.21.0->accelerate) (4.13.0)
Requirement already satisfied: certifi in /opt/anaconda3/envs/vit-pr
oject/lib/python3.10/site-packages (from httpx<1,>=0.23.0->huggingfa
ce_hub>=0.21.0->accelerate) (2026.1.4)
Requirement already satisfied: httpcore==1.* in /opt/anaconda3/envs/
vit-project/lib/python3.10/site-packages (from httpx<1,>=0.23.0->hug
gingface_hub>=0.21.0->accelerate) (1.0.9)
Requirement already satisfied: idna in /opt/anaconda3/envs/vit-proje
ct/lib/python3.10/site-packages (from httpx<1,>=0.23.0->huggingface_
hub>=0.21.0->accelerate) (3.11)
Requirement already satisfied: h11>=0.16 in /opt/anaconda3/envs/vit-
project/lib/python3.10/site-packages (from httpcore==1.*->httpx<1,>=
0.23.0->huggingface_hub>=0.21.0->accelerate) (0.16.0)
Requirement already satisfied: networkx in /opt/anaconda3/envs/vit-p
roject/lib/python3.10/site-packages (from torch>=2.0.0->accelerate)
(3.4.2)
Requirement already satisfied: jinja2 in /opt/anaconda3/envs/vit-pro
ject/lib/python3.10/site-packages (from torch>=2.0.0->accelerate) (
3.1.6)
Requirement already satisfied: sympy==1.13.1 in /opt/anaconda3/envs/
vit-project/lib/python3.10/site-packages (from torch>=2.0.0->acceler
ate) (1.13.1)
```

```
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /opt/anaconda3/
envs/vit-project/lib/python3.10/site-packages (from sympy==1.13.1->t
orch>=2.0.0->accelerate) (1.3.0)
Requirement already satisfied: exceptiongroup>=1.0.2 in /opt/anacond
a3/envs/vit-project/lib/python3.10/site-packages (from anyio->httpx<
1,>=0.23.0->huggingface_hub>=0.21.0->accelerate) (1.3.1)
Requirement already satisfied: MarkupSafe>=2.0 in /opt/anaconda3/env
s/vit-project/lib/python3.10/site-packages (from jinja2->torch>=2.0.
0->accelerate) (3.0.2)
Requirement already satisfied: click>=8.2.1 in /opt/anaconda3/envs/v
it-project/lib/python3.10/site-packages (from typer->huggingface_hub
>=0.21.0->accelerate) (8.3.2)
Requirement already satisfied: shellingham>=1.3.0 in /opt/anaconda3/
envs/vit-project/lib/python3.10/site-packages (from typer->huggingfa
ce_hub>=0.21.0->accelerate) (1.5.4)
Requirement already satisfied: rich>=12.3.0 in /opt/anaconda3/envs/v
it-project/lib/python3.10/site-packages (from typer->huggingface_hub
>=0.21.0->accelerate) (15.0.0)
Requirement already satisfied: annotated-doc>=0.0.2 in /opt/anaconda
3/envs/vit-project/lib/python3.10/site-packages (from typer->hugging
face_hub>=0.21.0->accelerate) (0.0.4)
Requirement already satisfied: markdown-it-py>=2.2.0 in /opt/anacond
a3/envs/vit-project/lib/python3.10/site-packages (from rich>=12.3.0-
>typer->huggingface_hub>=0.21.0->accelerate) (4.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /opt/anaco
nda3/envs/vit-project/lib/python3.10/site-packages (from rich>=12.3.
0->typer->huggingface_hub>=0.21.0->accelerate) (2.20.0)
Requirement already satisfied: mdurl~=0.1 in /opt/anaconda3/envs/vit
-project/lib/python3.10/site-packages (from markdown-it-py>=2.2.0->r
ich>=12.3.0->typer->huggingface_hub>=0.21.0->accelerate) (0.1.2)
Note: you may need to restart the kernel to use updated packages.
```

In [20]:

```python
from transformers import TrainingArguments, Trainer
from sklearn.metrics import accuracy_score, f1_score

def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = np.argmax(logits, axis=-1)
    acc = accuracy_score(labels, preds)
    f1 = f1_score(labels, preds, average="weighted")
    return {"accuracy": acc, "f1": f1}

training_args = TrainingArguments(
    output_dir="./vit-beans",
    num_train_epochs=4,
    per_device_train_batch_size=8,
    per_device_eval_batch_size=8,
    eval_strategy="epoch",
    save_strategy="epoch",
    load_best_model_at_end=True,
    metric_for_best_model="accuracy",
    logging_steps=10,
    learning_rate=2e-5,
    warmup_steps=50,
    remove_unused_columns=False,
    report_to="none",
```

```
    use_cpu=True            # ← clean CPU training
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=encoded_dataset["train"],
    eval_dataset=encoded_dataset["validation"],
    compute_metrics=compute_metrics,
)

print("Trainer configured and ready!")
```

```
Trainer configured and ready!
```

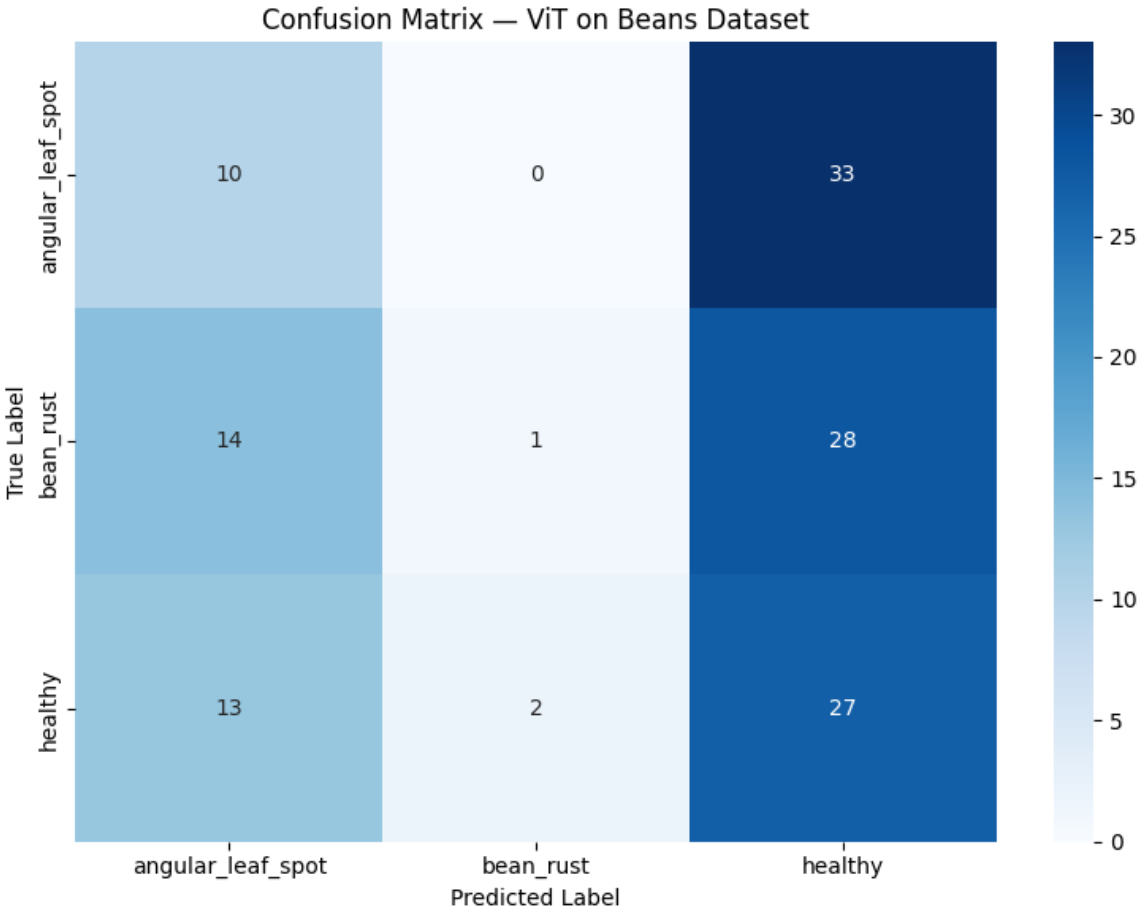
In [21]:
```
!pip install seaborn
```

```
Requirement already satisfied: seaborn in /opt/anaconda3/lib/python
3.12/site-packages (0.13.2)
Requirement already satisfied: numpy!=1.24.0,>=1.20 in /opt/anaconda
3/lib/python3.12/site-packages (from seaborn) (1.26.4)
Requirement already satisfied: pandas>=1.2 in /opt/anaconda3/lib/pyt
hon3.12/site-packages (from seaborn) (2.2.2)
Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in /opt/anaco
nda3/lib/python3.12/site-packages (from seaborn) (3.8.4)
Requirement already satisfied: contourpy>=1.0.1 in /opt/anaconda3/li
b/python3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (
1.2.0)
Requirement already satisfied: cycler>=0.10 in /opt/anaconda3/lib/py
thon3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (0.1
1.0)
Requirement already satisfied: fonttools>=4.22.0 in /opt/anaconda3/l
ib/python3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn)
(4.51.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /opt/anaconda3/l
ib/python3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn)
(1.4.4)
Requirement already satisfied: packaging>=20.0 in /opt/anaconda3/li
b/python3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (
23.2)
Requirement already satisfied: pillow>=8 in /opt/anaconda3/lib/pytho
n3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (12.0.0)
Requirement already satisfied: pyparsing>=2.3.1 in /opt/anaconda3/li
b/python3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (
3.0.9)
Requirement already satisfied: python-dateutil>=2.7 in /opt/anaconda
3/lib/python3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seabor
n) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /opt/anaconda3/lib/py
thon3.12/site-packages (from pandas>=1.2->seaborn) (2024.1)
Requirement already satisfied: tzdata>=2022.7 in /opt/anaconda3/lib/
python3.12/site-packages (from pandas>=1.2->seaborn) (2025.2)
Requirement already satisfied: six>=1.5 in /opt/anaconda3/lib/python
3.12/site-packages (from python-dateutil>=2.7->matplotlib!=3.6.1,>=
3.4->seaborn) (1.16.0)
```

In [22]:
```
from sklearn.metrics import confusion_matrix, classification_report
```

```python
import matplotlib.pyplot as plt
import seaborn as sns

# Get predictions on test set
outputs = trainer.predict(encoded_dataset["test"])
preds = np.argmax(outputs.predictions, axis=-1)
true_labels = outputs.label_ids

# Print report
print(classification_report(true_labels, preds, target_names=labels

# Plot confusion matrix
cm = confusion_matrix(true_labels, preds)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=labels, yticklabels=labels)
plt.title("Confusion Matrix - ViT on Beans Dataset")
plt.ylabel("True Label")
plt.xlabel("Predicted Label")
plt.tight_layout()
plt.show()
```

```
                   precision    recall  f1-score   support

angular_leaf_spot       0.27      0.23      0.25        43
        bean_rust       0.33      0.02      0.04        43
          healthy       0.31      0.64      0.42        42

         accuracy                           0.30       128
        macro avg       0.30      0.30      0.24       128
     weighted avg       0.30      0.30      0.23       128
```



In [23]:
```
model.save_pretrained("./vit-beans-final")
processor.save_pretrained("./vit-beans-final")

print("Model saved to ./vit-beans-final")
print("\nFiles saved:")
for f in os.listdir("./vit-beans-final"):
    print(f"  - {f}")
```

```
Writing model shards: 100%|██████████████████████| 1/1 [00:00<00:0
0,  1.74it/s]
Model saved to ./vit-beans-final

Files saved:
  - model.safetensors
  - config.json
  - preprocessor_config.json
```

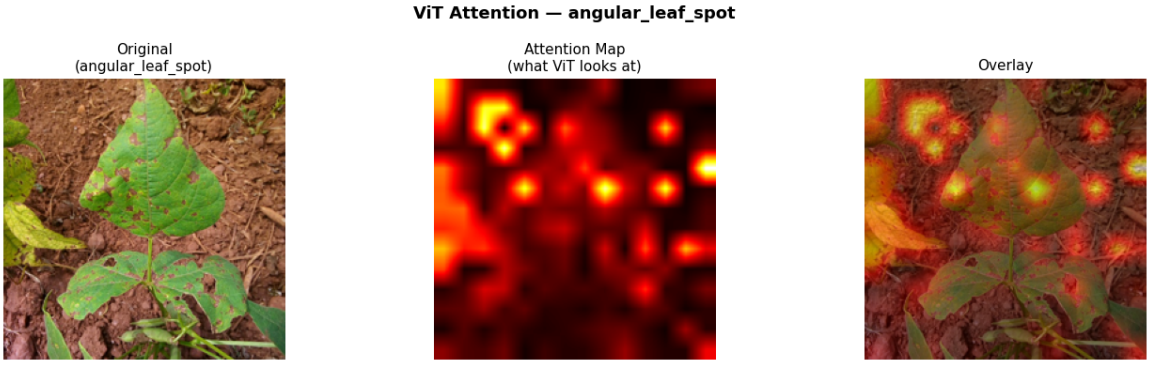
Attention Map Visualization

In [25]:
```
from transformers import ViTForImageClassification
import torch
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image

# Reload model with eager attention
model_attn = ViTForImageClassification.from_pretrained(
```

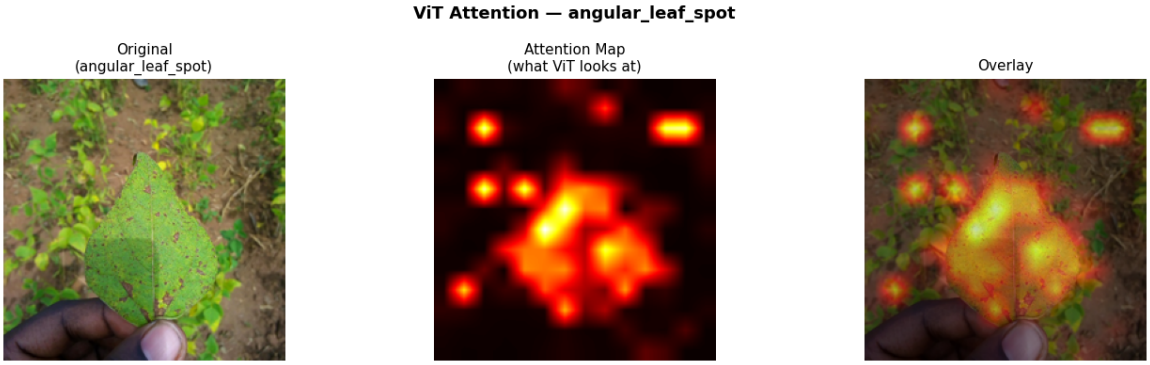
```
    "./vit-beans-final",
    attn_implementation="eager",
    ignore_mismatched_sizes=True
)
model_attn.eval()
print("Model reloaded with eager attention!")

def visualize_attention(model, processor, image, label_name):
    model.eval()
    inputs = processor(images=image.convert("RGB"), return_tensors=

    with torch.no_grad():
        outputs = model(**inputs, output_attentions=True)

    # Check attentions exist
    if outputs.attentions is None or len(outputs.attentions) == 0:
        print("No attentions returned")
        return

    # Last layer, average across all heads
    attention = outputs.attentions[-1]       # (1, heads, tokens, t
    attention = attention.mean(dim=1)        # (1, tokens, tokens)

    # CLS token row = how much CLS attends to each patch
    cls_attention = attention[0, 0, 1:]      # skip CLS token itsel

    # Reshape to 14x14 spatial grid
    size = int(cls_attention.shape[0] ** 0.5)
    cls_attention = cls_attention.reshape(size, size).numpy()

    # Upscale attention map to 224x224
    attn_resized = np.array(
        Image.fromarray(cls_attention).resize((224, 224), Image.BIL
    )

    # Plot: original | attention map | overlay
    fig, axes = plt.subplots(1, 3, figsize=(14, 4))
    img_resized = image.convert("RGB").resize((224, 224))

    axes[0].imshow(img_resized)
    axes[0].set_title(f"Original\n({label_name})", fontsize=11)
    axes[0].axis("off")

    axes[1].imshow(attn_resized, cmap="hot")
    axes[1].set_title("Attention Map\n(what ViT looks at)", fontsiz
    axes[1].axis("off")

    axes[2].imshow(img_resized)
    axes[2].imshow(attn_resized, cmap="hot", alpha=0.5)
    axes[2].set_title("Overlay", fontsize=11)
    axes[2].axis("off")

    plt.suptitle(f"ViT Attention — {label_name}", fontsize=13, font
    plt.tight_layout()
    plt.savefig(f"attention_{label_name.replace(' ', '_')}.png",
                dpi=150, bbox_inches="tight")
```

```
    plt.show()
    print(f"Saved: attention_{label_name.replace(' ', '_')}.png")

# Visualize 3 test samples
for i in range(3):
    sample = dataset["test"][i]
    visualize_attention(model_attn, processor, sample["image"], lab
```

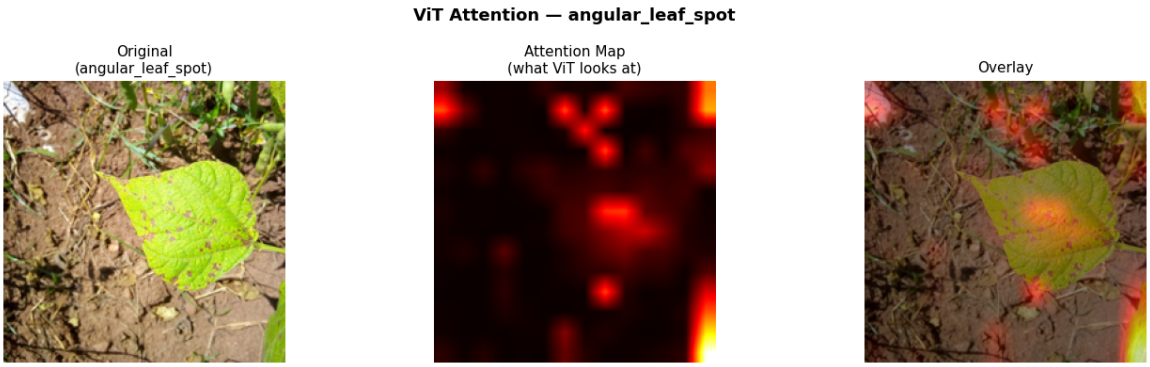
```
Loading weights: 100%|▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰| 200/200 [00:00<00:00,
5300.09it/s]
Model reloaded with eager attention!
```




```
Saved: attention_angular_leaf_spot.png
```




```
Saved: attention_angular_leaf_spot.png
```




```
Saved: attention_angular_leaf_spot.png
```

ViT vs ResNet Comparison

In [29]:

```
import torchvision.models as tv_models
import torchvision.transforms as transforms
from torch.utils.data import DataLoader
import time

print("Training ResNet18 on same dataset for comparison...\n")
```

```python
# ResNet transform
resnet_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406],
                         [0.229, 0.224, 0.225])
])

# Build simple torch datasets
class BeansDataset(torch.utils.data.Dataset):
    def __init__(self, hf_dataset, transform):
        self.data = hf_dataset
        self.transform = transform
    def __len__(self):
        return len(self.data)
    def __getitem__(self, idx):
        img = self.data[idx]["image"].convert("RGB")
        label = self.data[idx]["labels"]
        return self.transform(img), label

train_ds = BeansDataset(dataset["train"], resnet_transform)
val_ds   = BeansDataset(dataset["validation"], resnet_transform)
test_ds  = BeansDataset(dataset["test"], resnet_transform)

train_loader = DataLoader(train_ds, batch_size=16, shuffle=True)
val_loader   = DataLoader(test_ds,  batch_size=16)

# Load pretrained ResNet18
resnet = tv_models.resnet18(weights=tv_models.ResNet18_Weights.DEFA
resnet.fc = torch.nn.Linear(resnet.fc.in_features, 3)
resnet = resnet.to("cpu")

optimizer = torch.optim.Adam(resnet.parameters(), lr=2e-4)
criterion = torch.nn.CrossEntropyLoss()

# Train for 4 epochs
start = time.time()
for epoch in range(4):
    resnet.train()
    total_loss = 0
    for imgs, lbls in train_loader:
        optimizer.zero_grad()
        out = resnet(imgs)
        loss = criterion(out, lbls)
        loss.backward()
        optimizer.step()
        total_loss += loss.item()
    print(f"Epoch {epoch+1}/4 — Loss: {total_loss/len(train_loader)

resnet_time = time.time() - start

# Evaluate ResNet
resnet.eval()
all_preds, all_labels = [], []
with torch.no_grad():
```

```
    for imgs, lbls in val_loader:
        out = resnet(imgs)
        preds = out.argmax(dim=1)
        all_preds.extend(preds.numpy())
        all_labels.extend(lbls.numpy())

resnet_acc = accuracy_score(all_labels, all_preds)
print(f"\nResNet18 Accuracy: {resnet_acc:.4f}")
```

```
Training ResNet18 on same dataset for comparison...

Downloading: "https://download.pytorch.org/models/resnet18-f37072fd.
pth" to /Users/ayshairam/.cache/torch/hub/checkpoints/resnet18-f3707
2fd.pth
100%|[redacted]| 44.7M/44.7M [00:09<00:00, 5.12M
B/s]
Epoch 1/4 — Loss: 0.3439
Epoch 2/4 — Loss: 0.0999
Epoch 3/4 — Loss: 0.0351
Epoch 4/4 — Loss: 0.0468

ResNet18 Accuracy: 0.9375
```

Get ViT accuracy from earlier evaluation

In [33]:

```
import torchvision.models as tv_models
import torchvision.transforms as transforms
from torch.utils.data import DataLoader
import time

print("Training ResNet18 on same dataset for comparison...\n")

resnet_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406],
                         [0.229, 0.224, 0.225])
])

class BeansDataset(torch.utils.data.Dataset):
    def __init__(self, hf_dataset, transform):
        self.data = hf_dataset
        self.transform = transform
    def __len__(self):
        return len(self.data)
    def __getitem__(self, idx):
        img = self.data[idx]["image"].convert("RGB")
        label = self.data[idx]["labels"]
        return self.transform(img), label
```

```
train_ds = BeansDataset(dataset["train"], resnet_transform)
test_ds  = BeansDataset(dataset["test"],  resnet_transform)

train_loader = DataLoader(train_ds, batch_size=16, shuffle=True)
val_loader   = DataLoader(test_ds,  batch_size=16)

resnet = tv_models.resnet18(weights=tv_models.ResNet18_Weights.DEFA
resnet.fc = torch.nn.Linear(resnet.fc.in_features, 3)
resnet = resnet.to("cpu")

optimizer = torch.optim.Adam(resnet.parameters(), lr=2e-4)
criterion = torch.nn.CrossEntropyLoss()

start = time.time()
for epoch in range(4):
    resnet.train()
    total_loss = 0
    for imgs, lbls in train_loader:
        optimizer.zero_grad()
        out = resnet(imgs)
        loss = criterion(out, lbls)
        loss.backward()
        optimizer.step()
        total_loss += loss.item()
    print(f"Epoch {epoch+1}/4 — Loss: {total_loss/len(train_loader)

resnet_time = time.time() - start
print(f"\nResNet training time: {resnet_time/60:.1f} minutes")

# Evaluate
resnet.eval()
all_preds, all_labels = [], []
with torch.no_grad():
    for imgs, lbls in val_loader:
        out = resnet(imgs)
        preds = out.argmax(dim=1)
        all_preds.extend(preds.numpy())
        all_labels.extend(lbls.numpy())

resnet_acc = accuracy_score(all_labels, all_preds)
print(f"ResNet18 Accuracy: {resnet_acc:.1%}")
print("\nresnet_acc is ready — now run the comparison chart cell!")
```

```
Training ResNet18 on same dataset for comparison...

Epoch 1/4 — Loss: 0.3586
Epoch 2/4 — Loss: 0.0831
Epoch 3/4 — Loss: 0.0706
Epoch 4/4 — Loss: 0.0615

ResNet training time: 6.8 minutes
ResNet18 Accuracy: 97.7%

resnet_acc is ready — now run the comparison chart cell!
```

In [ ]: